



PHIẾU GIAO BÀI TẬP

**BÀI 6 – THỰC HÀNH LẬP TRÌNH KẾT NỐI
CƠ SỞ DỮ LIỆU**

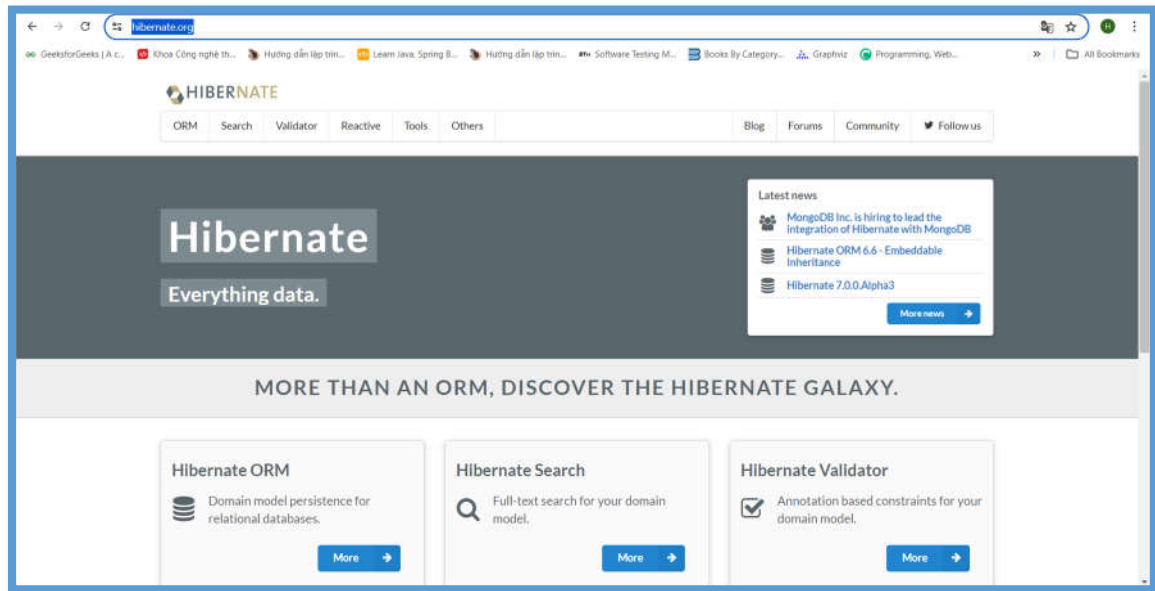
MỤC LỤC

6.1 Xây dựng một Hibernate Project	2
6.1.1 Tải và cài đặt.....	2
6.1.2 Lựa chọn cấu hình	2
6.1.3 Cài đặt và sử dụng	3
6.2 Xây dựng JPA project trong Eclipse	13
6.3 Viết các kịch bản chạy thử truy cập CSDL	13
6.4 Viết các kịch bản kiểm thử với công cụ JMeter	14

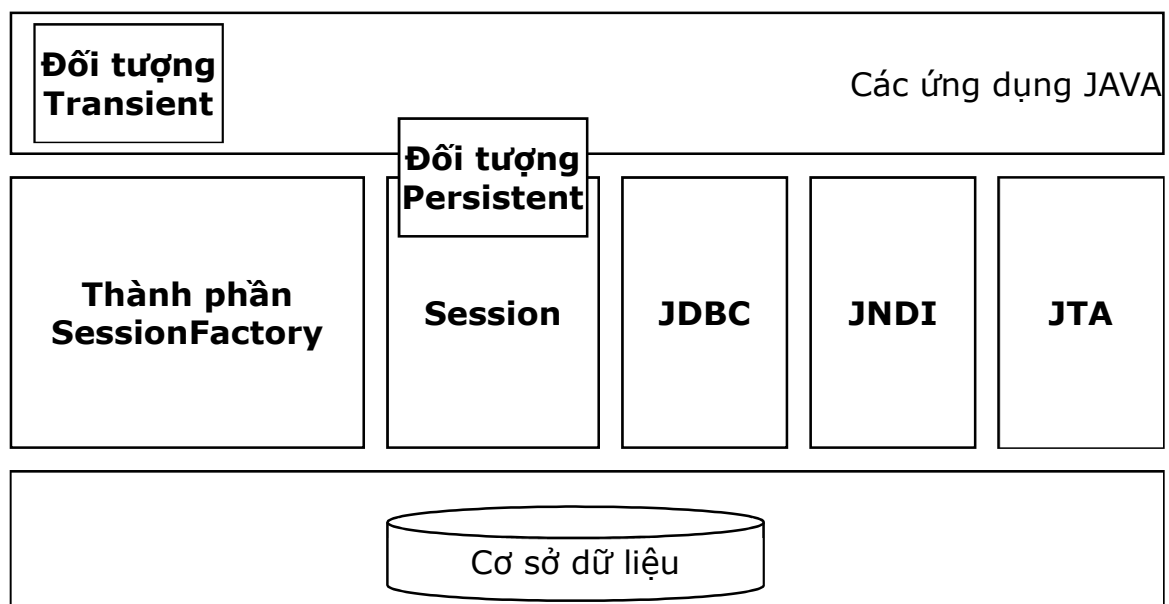
6.1 Xây dựng một Hibernate Project

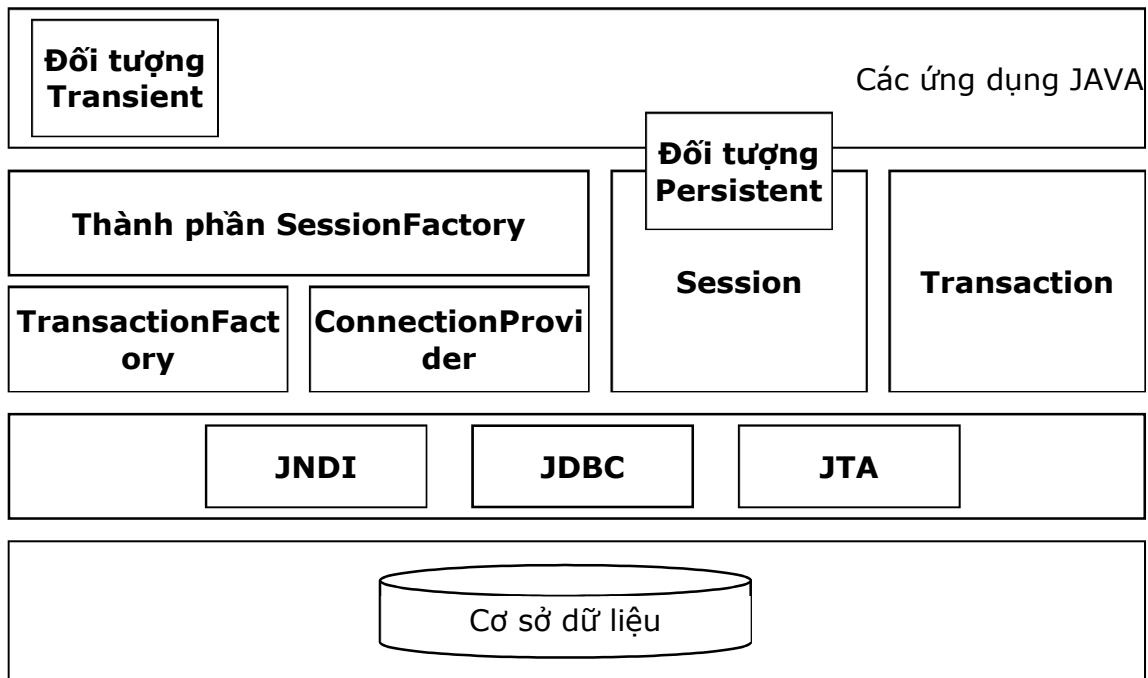
6.1.1 Tải và cài đặt

Vào liên kết: <https://hibernate.org/>



6.1.2 Lựa chọn cấu hình



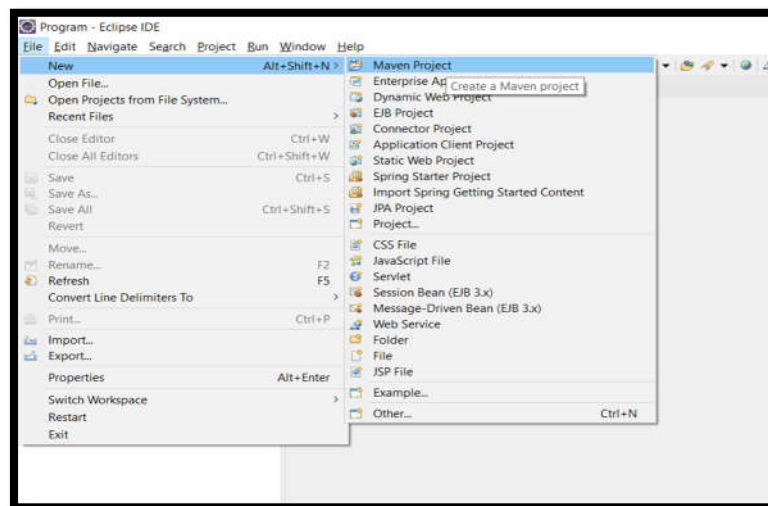


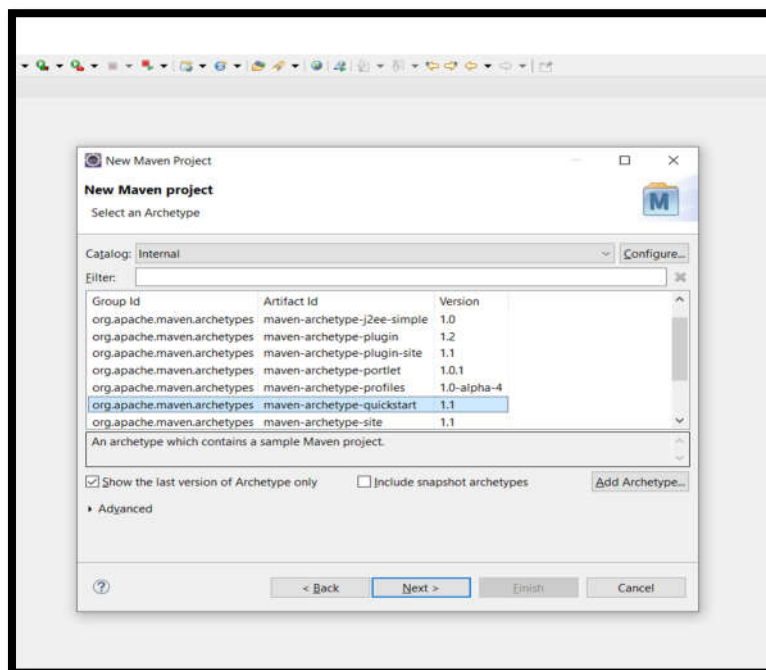
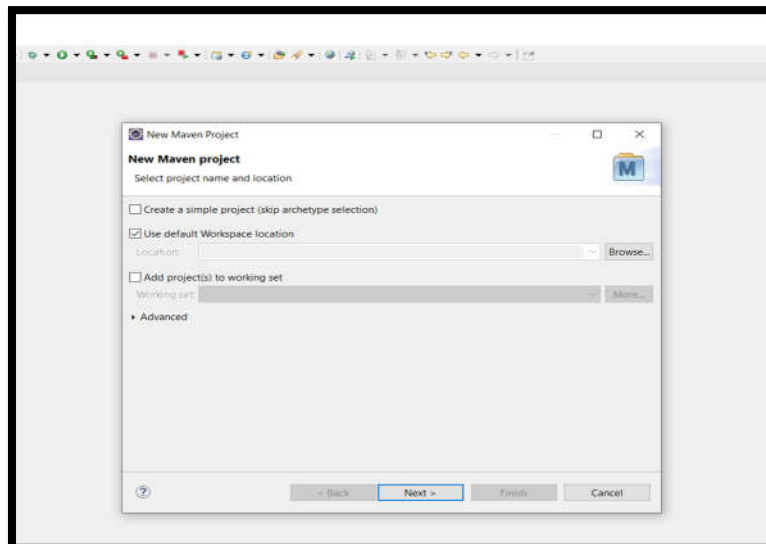
6.1.3 Cài đặt và sử dụng

Để sử dụng Hibernate, chúng ta sẽ lần lượt thực hiện các bước sau:

- Tạo maven project.

Trong Eclipse chúng ta thực hiện theo





- b) Khai báo thư viện Hibernate và thư viện JDBC ứng với loại database cần sử dụng.

Sau khi tạo maven project và trong file pom.xml khai báo thư viện Hibernate 5, và thư viện JDBC tương ứng cho các loại Database khác nhau MySQL, PostgreSQL, Oracle, SQL Server, ...

Trong bài này, chúng ta sẽ sử dụng database MySQL. Nội dung file pom.xml có thể như sau:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```



```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>hau1.fit</groupId>
  <artifactId>MVExamples</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>Examples</name>

  <properties>
    <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>

  <dependencies>

    <!--
https://mvnrepository.com/artifact/org.projectlombok/lombok -
->
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>1.18.10</version>
    </dependency>

    <!--
https://mvnrepository.com/artifact/org.hibernate/hibernate-
core -->
    <dependency>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>5.4.7.Final</version>
    </dependency>

    <!-- MySQL -->
    <!-- https://mvnrepository.com/artifact/mysql/mysql-
connector-java -->
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>5.1.49</version>
    </dependency>

    <!-- Unit Test -->
    <dependency>
      <groupId>junit</groupId>
```

```
<artifactId>junit</artifactId>
<version>3.8.1</version>
<scope>test</scope>
</dependency>
</dependencies>
```

```
</project>
```

c) Tạo các Entity và các mapping.

Chúng ta sử dụng lại các Entity ở Bài số 5, các em chuyển sang Project này để làm việc

```
package hui.fit;

import java.util.Date;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

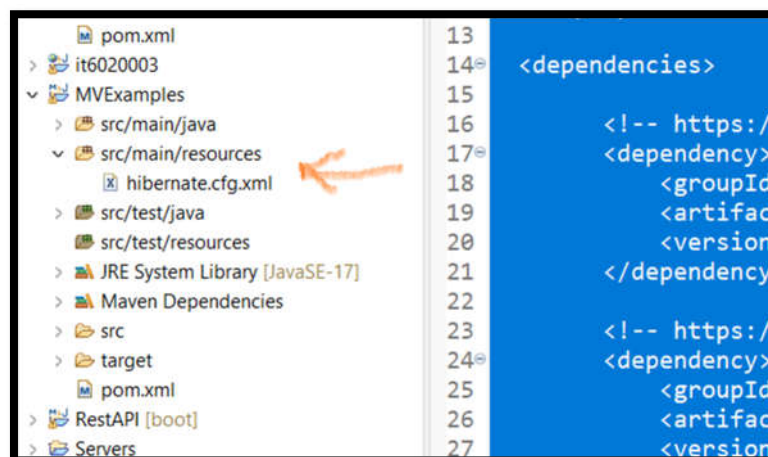
import lombok.*;

@Data
@Entity
@Table(name = "tbluser")
public class UserObject {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long user_id;
    @Column
    private String user_fullname;
    @Column
```

```
private String user_name;
@Column
private String user_pass;
@Column(name = "user_created_date")
private Date createdAt;
@Column(name = "user_last_modified")
private Date modifiedAt;
@Column
private String user_email;
@Column
private Long user_parent_id;
}
```

d) Tạo file cấu hình Hibernate.

Để làm việc với Hibernate, chúng ta cần phải có một file cấu hình. Trong file này, chúng ta sẽ khai báo loại database mà chúng ta sẽ sử dụng, những thông tin cần thiết để Hibernate có thể kết nối đến database đó và những Java object mà chúng ta đã ánh xạ từ java object (entity) với các table trong database.



Nội dung file hibernate.cfg.xml như dưới đây

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC "-//
//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
```



```
<hibernate-configuration>
  <session-factory>
    <!-- Database setting -->
    <property
name="connection.driver_class">com.mysql.jdbc.Driver</property>
    <property
name="connection.url">jdbc:mysql://localhost:3309/it6020003_data?serverTimezone=UTC&useUnicode=true&characterEncoding=utf8</property>
    <property
name="connection.username">it6020003_huyhq</property>
    <property
name="connection.password">213456!@#</property>

    <!-- JDBC connection pool (use the built-in) -->
    <property name="connection.pool_size">10</property>

    <!-- SQL dialect -->
    <property
name="dialect">org.hibernate.dialect.MySQL5Dialect</property>

    <!-- Enable Hibernate's automatic session context
management -->
    <property
name="current_session_context_class">thread</property>

    <!-- Disable the second-level cache -->
    <property
name="cache.provider_class">org.hibernate.cache.internal.NoCacheProvider</property>

    <!-- Show all executed SQL to console -->
    <property name="show_sql">true</property>

    <!-- Entity mapping -->
    <mapping class="hau.fit.UserObject" />

  </session-factory>
</hibernate-configuration>
```

e) Tạo đối tượng SessionFactory

SessionFactory là một interface giúp tạo ra session kết nối đến database bằng cách đọc các cấu hình trong Hibernate configuration.



SessionFactory là đối tượng hoạt động cường độ cao (heavy weight object) và được sử dụng thường xuyên nên chúng ta sẽ tạo một class singleton HibernateUtils để sử dụng sau này.

Nội dung chi tiết HibernateUtils.java

```
package haui.fit.utils;

import org.hibernate.SessionFactory;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;
import org.hibernate.service.ServiceRegistry;

public class HibernateUtils {

    private static final SessionFactory sessionFactory =
        buildSessionFactory();

    private HibernateUtils() {
        super();
    }

    private static SessionFactory buildSessionFactory() {
        // Load hibernate.cfg.xml from resource folder by default
        ServiceRegistry serviceRegistry = new
        StandardServiceRegistryBuilder()
            .configure()
            .build();

        Metadata metadata = new
        MetadataSources(serviceRegistry).getMetadataBuilder().build()
        ;
    }
}
```



```
        return metadata.getSessionFactoryBuilder().build();
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }

    public static void close() {
        getSessionFactory().close();
    }
}
```

f) Sử dụng SessionFactory để thực hiện các câu lệnh truy vấn đến database.
Mã nguồn hoàn chỉnh để thực hiện

```
package hui.fit;
import java.util.Date;
import java.util.List;
import org.hibernate.Session;
import hui.fit.utils.*;
public class HibernateExample1 {
    public static void main(String[] args) {
        try (Session session =
HibernateUtils.getSessionFactory().openSession();) {
            // Begin a unit of work
            session.beginTransaction();
            // Insert user
            Date currentDate = new Date();
```



```
UserObject user1 = new UserObject();

user1.setUser_fullname("Hoàng Quang Huy");
user1.setUser_name("huyhq456");
user1.setUser_pass("123456");
user1.setCreatedAt(currentDate);
user1.setModifiedAt(currentDate);
user1.setUser_email("huyhq@hau.edu.vn");
user1.setUser_parent_id((long)20);

Long userId = (Long) session.save(user1);
System.out.println("User id = " + userId);

//          // Count user from database
//          Long numberOfUser =
session.createQuery("SELECT COUNT(id) FROM UserObject",
Long.class).uniqueResult();
//          System.out.println("Number of user in
database: " + numberOfUser);
//
//          // Get user by id
//          UserObject savedUser =
session.find(UserObject.class, userId);
//          System.out.println("savedUser: " + savedUser);
//          // Update user
//          savedUser.setUser_fullname("Hoàng Vương Bảo");
//          session.update(savedUser);
//          // Get users
```

```

        List<UserObject> users =
session.createQuery("FROM UserObject",
UserObject.class).list();

        users.forEach(System.out::println);
//
// Delete user
//
session.delete(savedUser);
//
// Count user from database
//
numberOfUser = session.createQuery("SELECT
COUNT(id) FROM UserObject", Long.class).uniqueResult();
//
System.out.println("Number of user in
database: " + numberOfUser);

        // Commit the current resource transaction,
writing any unflushed changes to the
// database.

session.getTransaction().commit();

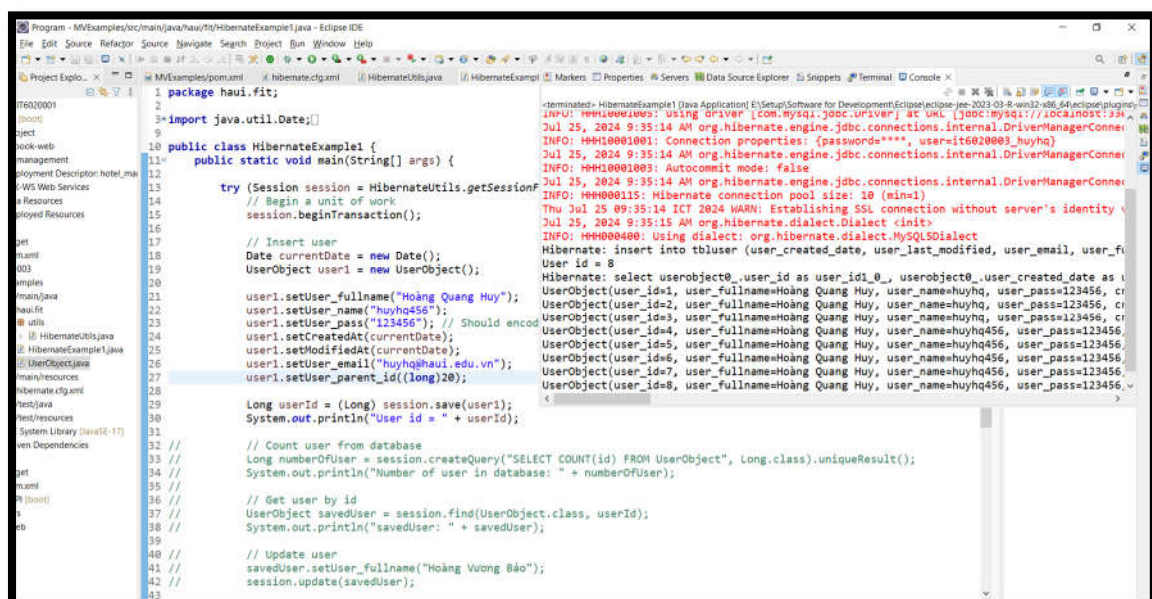
    }

}

}

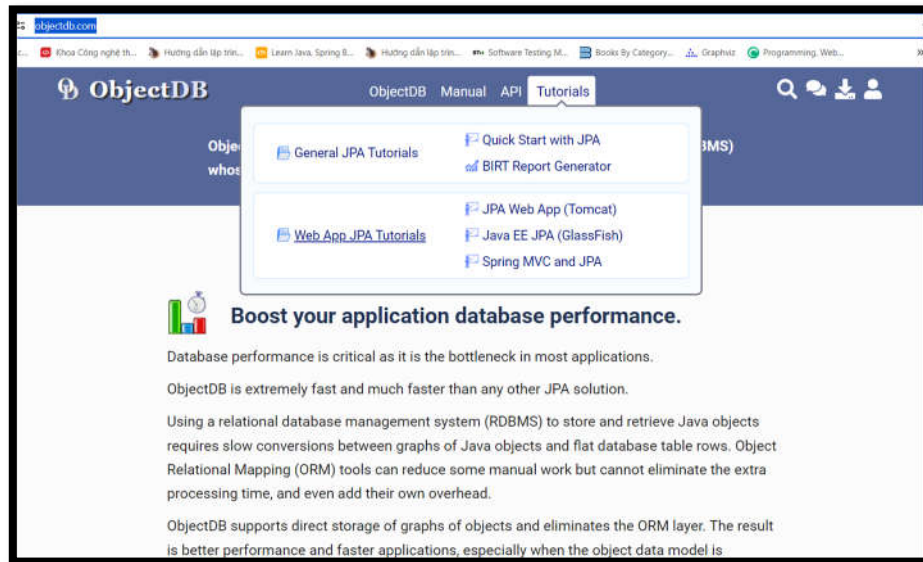
```

Kết quả chạy như hình dưới



6.2 Xây dựng JPA project trong Eclipse

Truy cập trang <https://www.objectdb.com/> để thực hiện phát triển một JPA Project



Các yêu cầu cụ thể được chỉ rõ trong liên kết trên.

Xây dựng chương trình đã được cung cấp dữ liệu ở các bước trước đây bằng JPA Project này. Đóng gói và chạy thử nghiệm.

6.3 Viết các kịch bản chạy thử truy cập CSDL

Kịch bản chạy thử nghiệm cho mô đun User giả sử như phương thức main sau

```
public static void main(String[] args) {  
    // Bộ quản lý kết nối  
    ConnectionPool cp = new ConnectionPoolImpl();  
  
    // Đối tượng thực thi chức năng  
    UserFunction uf = new UserFunctionImpl(cp);  
  
    // Đối tượng người sử dụng mới  
    UserObject nItem = new UserObject();  
    nItem.setUser_name("JavaNangcao");  
    nItem.setUser_email("huyhq@hauai.edu.vn");  
    nItem.setUser_created_date("10/05/24");  
    nItem.setUser_fullname("Hoang Quang Huy");  
    nItem.setUser_parent_id(20);  
    nItem.setUser_pass("123456");  
  
    boolean results = uf.addUser(nItem);  
    if(!results) {  
        System.out.print("-----KHÔNG THÀNH CÔNG-----\n");  
    }  
}
```

```
// Danh sách đối tượng
ArrayList<UserObject> list = uf getUsers(null, 0, (byte)
10);

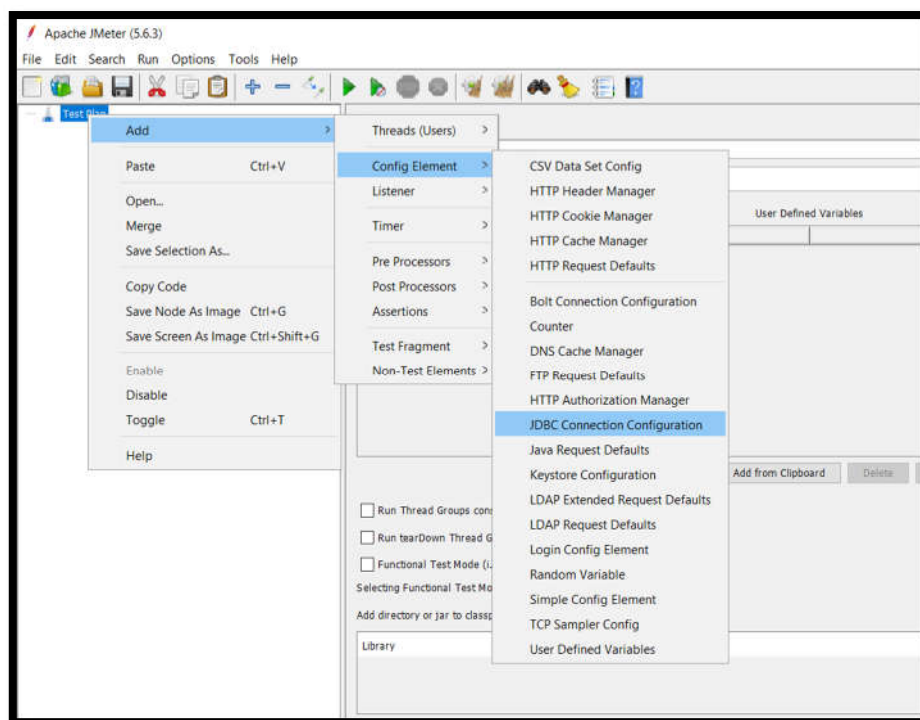
list.forEach(u -> {
    System.out.println(u);
});

}
```

Các kịch bản chạy khác cho các mô đun khác yêu cầu tương tự

6.4 Viết các kịch bản kiểm thử với công cụ JMeter

- Cài đặt công cụ JMeter
- Cấu hình kiểm thử như hình minh họa sau cho các mô đun và toàn chương trình



JDBC Request

Name: JDBC Request

Comments:

Variable Name Bound to Pool

Variable Name:

SQL Query

Query Type: Select Statement

Query:
select test_result from test_tbl where id = 1
SQL query

Parameter values:

Parameter types:

Variable names:

Result variable name:

YÊU CẦU CHUNG:

- ☞ Chương trình cài đặt được quản lý theo từng nhiệm vụ
- ☞ Chuẩn bị dữ liệu đầy đủ và chạy demo
- ☞ Viết chú thích cho từng khối lệnh
- ☞ Viết báo cáo ngắn mô tả các kỹ thuật xử lý SQL